

# Chapter 9

## Testing Object-Oriented Software



# Outline

- Issues in testing object-oriented software (section 9.5)
- Unit testing frameworks
- Mock objects
- Data flow testing
- Slice-based testing
- Object-oriented Complexity Metrics



# Issues in testing object-oriented software

- (Much of this discussion depends on the java code in Chapter 9)
- Implications of Composition and Encapsulation
- Implications of Inheritance
- Implications of Polymorphism



# Composition

- Central design strategy of all o-o software
- A bottom-up process
- A class might be composed with previously developed classes. BUT
- The “Structured Programming” considerations apply...
  - Classes must be very cohesive, and
  - Loosely coupled
- Highly cohesive classes promote re-use
- Highly coupled classes require more integration testing



# Encapsulation

- A fundamental goal of o-o design is that a class “encapsulates” the methods and data that “belong” together.
- A well-encapsulated class
  - Provides a distinct service to other classes
  - Does not depend on other classes
  - Does not interfere with functionality of other classes
- All inter-class interaction is accomplished with messages.

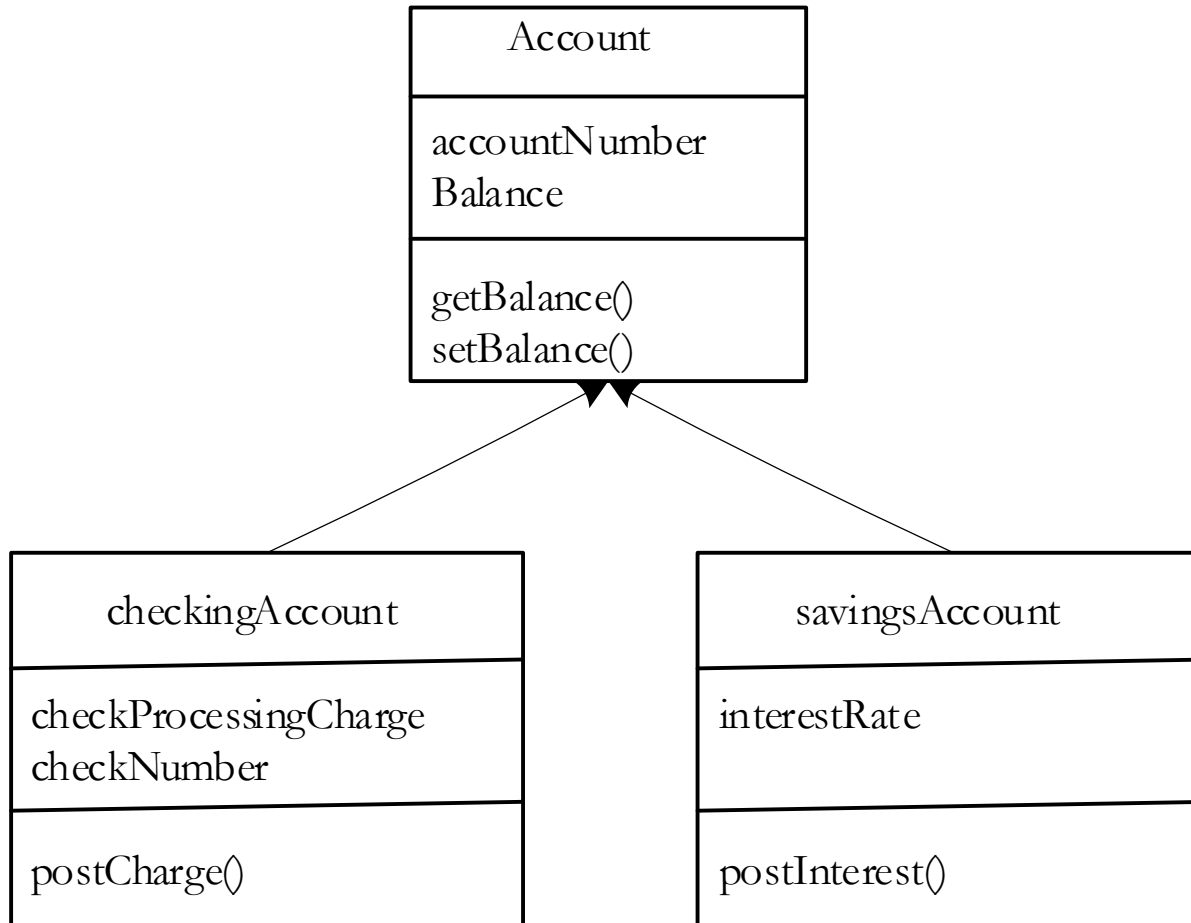


# Inheritance

- One of the guiding precepts of o-o design and programming is to reduce (eliminate?) repetition.
- The Inheritance mechanism accomplishes this by moving repeated data and methods to a higher level, abstract class.
- BUT abstract classes are not executable, hence not testable.
- One answer: Bob Binder's concept of flattened classes.
- (See the next two slides)



# Inheritance Example



# Flattened Classes

| checkingAccount                                                  |
|------------------------------------------------------------------|
| accountNumber<br>Balance<br>checkProcessingCharge<br>checkNumber |
| getBalance()<br>setBalance()<br>postCharge()                     |

| savingsAccount                                 |
|------------------------------------------------|
| accountNumber<br>Balance<br>interestRate       |
| getBalance()<br>setBalance()<br>postInterest() |





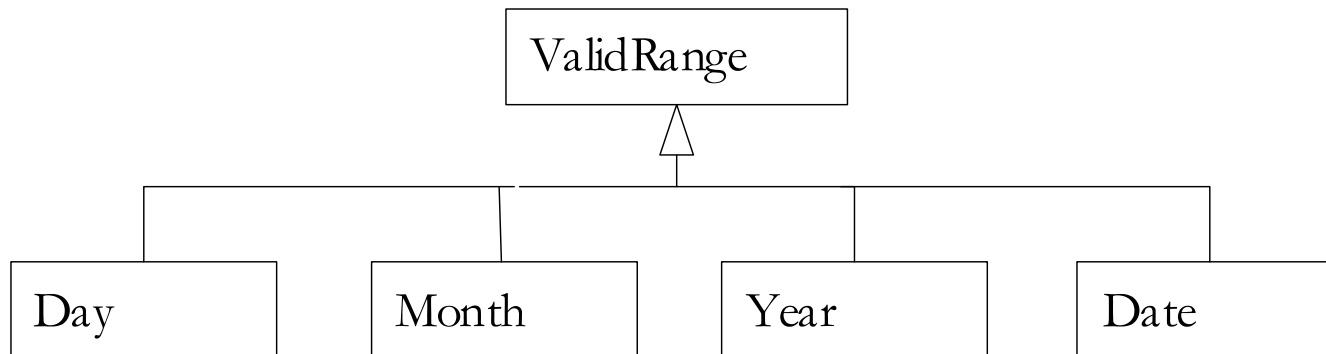
# New Issue

- Flattened classes will not be part of a delivered system, so their testing is somewhat artificial.
- One workaround: add special purpose methods just for testing.
- (Somewhat analogous to instrumented code)
- What if the test methods are faulty?
  - False positive? (test passes when it shouldn't)
  - False negative? (test fails when it shouldn't)
- The false positive/negative terms are confusing. Better choice?
  - False pass
  - False fail



# Polymorphism

- Same method applies to different objects.
- Must be covered in class-as-a-unit testing.
- (See listing 9.6)
- Figure 9.5 repeated here.



# In Listing 9.6...

- The Date class entirely overrides parent functionality
- The Day and Month classes override the parent functionality...
- BUT then use it as part of their own implementation in lines 86-89 and 145-148, respectively.
- The Year class uses the parent functionality as-is, without overriding in its own class definition.



# Unit Testing Frameworks

- Earliest framework? SUnit for Smalltalk
  - Essential to the Agile Manifesto
  - Automatic execution of unit level tests
- Inspired JUnit (and the Xunit concept)
- Nearly 100 programming languages have Xunit frameworks.
  - JUnit for Java
  - NUnit for C
  - CPPUnit for C++
- See SimpleDate example in Section 9.1.2
- Basic mechanism? Assert statements inline with code.



# Object-Oriented Complexity Metrics

- Chidamber/Kemerer o-o complexity metrics
  - WMC—Weighted Methods per Class
  - DIT—Depth of Inheritance Tree
  - NOC—Number of Child Classes
  - CBO—Coupling between Classes
  - RFC—Response for Class
  - LCOM—Lack of Cohesion on Methods
- These are detailed in section 9.4
- Exercises: Which of these can be derived from...
  - UML diagrams?
  - Program Graphs?
  - Source code?



# Assert Syntax

- Can assert True or False
- Refer to Class.Method
- Can pass values to the method being tested.
- `assertTrue(simpleDate.rangesOK(2, 29, 2000));`
- `assertFalse(simpleDate.rangesOK(2, 29, 2001));`



# Listing 9.2....

```
import org.junit.Test;
import static org.junit.Assert.*;

public class SimpleDateTest {

    @Test
    public void testWithDependency() {
        // This test has a dependency on SimpleDate.isLeap
        SimpleDate simpleDate = new SimpleDate(1, 1, 2000);

        assertTrue(simpleDate.rangesOK(2, 29, 2000)); // Valid due to leap year
        assertFalse(simpleDate.rangesOK(2, 29, 2001)); // Invalid due to leap year
    }
}
```



# Mock Objects

- Despite the name, some units require the presence of other units, either as sources of inputs or as destinations of outputs (messages).
- In Chapter 13, we will see the need for “Stubs” and “Drivers” for procedural code integration testing.
- Mock objects take the place of stubs and drivers for o-o software.
- See example in Listing 9.4, repeated on the next slide.
- Notice that the override acts as a driver for procedural code..
  - It contains the assertion inputs (2,29,2000) and (2,29,2001)
  - the expected outputs return true or return false





```
@Test
public void testWithoutDependencyManual() {
    // This test has no dependency on SimpleDate.isLeap
    SimpleDate simpleDate = new SimpleDate(1, 1, 2000) {
        @Override
        protected boolean isLeap(int year) {
            if(2000 == year)
                return true;
            else if(2001 == year)
                return false;
            else
                throw new IllegalArgumentException("No Mock for year " + year);
        }
    };

    assertTrue(simpleDate.rangesOK(2, 29, 2000)); // Valid due to leap year
    assertFalse(simpleDate.rangesOK(2, 29, 2001)); // Valid due to leap year
}
```



# Dataflow Testing

- An overloaded term, do not confuse with dataflow diagrams.
- A better name might be Define/Use Testing
- Describes what happens to a variable between when it acquires a value and when that value is used.
- Elaborates on early COBOL concordance anomalies:
  - Variables that are defined but never used
  - Variables that are used before they are defined
  - Variables that are defined repeatedly before they are used



# Define/Use Definitions

- These definitions are based on a set  $V$  of variables in a program  $P$  with program graph  $G(P)$ . Usually  $V$  contains just one variable. The program  $P$  is written in a procedural programming language.
- Node  $n \in G(P)$  is a **defining node of the variable  $v \in V$** , written as  $DEF(v, n)$ , if and only if the value of the variable  $v$  is defined at the statement fragment corresponding to node  $n$ .
- Node  $n \in G(P)$  is a **usage node of the variable  $v \in V$** , written as  $USE(v, n)$ , if and only if the value of the variable  $v$  is used at the statement fragment corresponding to node  $n$ .
- A usage node  $USE(v, n)$  is a **predicate use** (denoted as P-use) if and only if the statement  $n$  is a predicate statement; otherwise,  $USE(v, n)$  is a **computation use**, (denoted C-use).



# Path Definitions

- $\text{Paths}(P)$  is the set of all paths in  $G(P)$ .
- A **definition-use path with respect to a variable  $v$**  (denoted du-path) is a path in  $\text{PATHS}(P)$  such that, for some  $v \in V$ , there are define and usage nodes  $\text{DEF}(v, m)$  and  $\text{USE}(v, n)$  such that  $m$  and  $n$  are the initial and final nodes of the path.
- A **definition-clear path with respect to a variable  $v$**  (denoted dc-path) is a definition-use path in  $\text{PATHS}(P)$  with initial and final nodes  $\text{DEF}(v, m)$  and  $\text{USE}(v, n)$  such that no other node in the path is a defining node of  $v$ .
- These definitions are used to define a set of test coverage metrics.

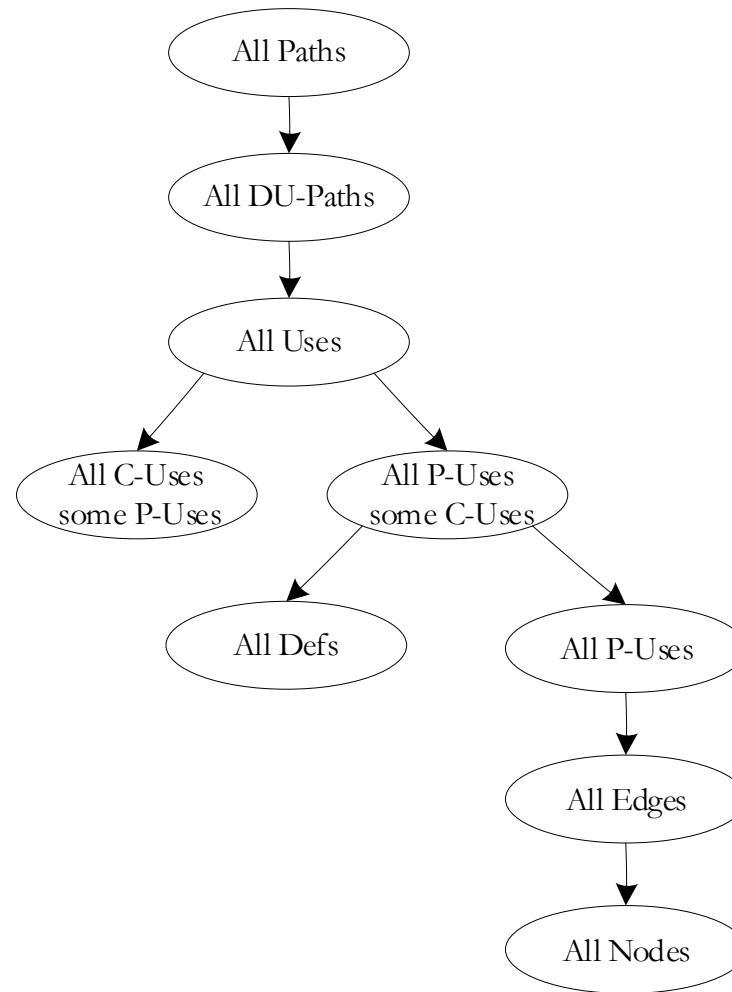


# Test Cover Definitions

- Let  $T$  be a set of test cases for program  $P$  written in a procedural programming language.
  - The set  $T$  satisfies the **All-Defs criterion** for the program  $P$  if and only if for every variable  $v \in V$ ,  $T$  contains definition-clear paths from every defining node of  $v$  to a use of  $v$ .
  - The set  $T$  satisfies the **All-Uses criterion** for the program  $P$  if and only if for every variable  $v \in V$ ,  $T$  contains definition-clear paths from every defining node of  $v$  to every use of  $v$ , and to the successor node of each  $USE(v, n)$ .
  - The set  $T$  satisfies the **All-P-Uses/Some C-Uses criterion** for the program  $P$  if and only if for every variable  $v \in V$ ,  $T$  contains definition-clear paths from every defining node of  $v$  to every predicate use of  $v$ ; and if a definition of  $v$  has no P-uses, a definition-clear path leads to at least one computation use.
  - The set  $T$  satisfies the **All-C-Uses/Some P-Uses criterion** for the program  $P$  if and only if for every variable  $v \in V$ ,  $T$  contains definition clear paths from every defining node of  $v$  to every computation use of  $v$ ; and if a definition of  $v$  has no C-uses, a definition-clear path leads to at least one predicate use.



# Lattice of Define/Use Test Coverage Metrics (Figure 9.1)



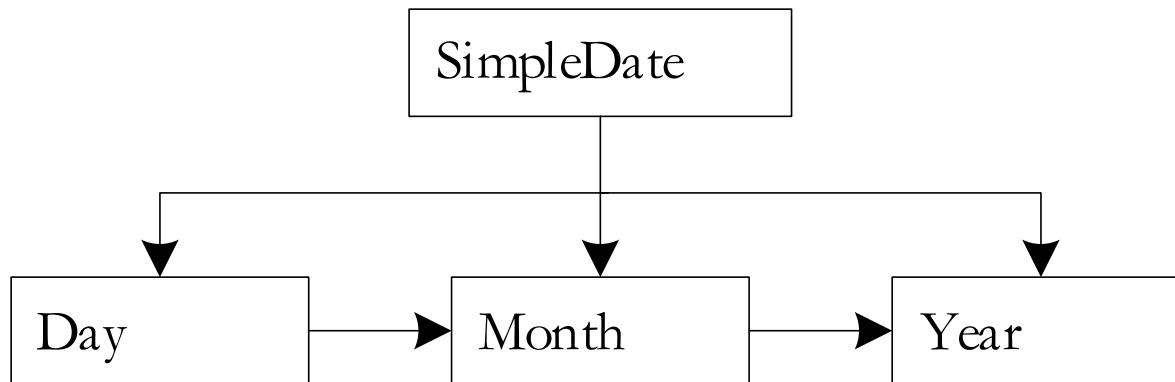
# Define/Use Testing Example

- If the program P is written in an object-oriented language, the earlier definitions must be revised. Define and Use nodes must consider changes due to
  - class aggregation,
  - inheritance,
  - dynamic binding, and
  - Polymorphism
- See Listing 9.5 for date related classes and a JUnit test class



# Define/Use Testing Example (continued)

- In Figure 9.2 (repeated here) we have
  - DateTest: A JUnit test class with a single test function (i.e., testSimple),
  - Date: A class representing date with a month, day, and year,
  - Day: A class representing a single day,
  - Month: A class representing a single month,
  - Year: A class representing a single year





# Define/Use Nodes in Listing 9.5

| Variable    | Defined at node | Used at nodes           |
|-------------|-----------------|-------------------------|
| Date.day    | 15              | 18, 21                  |
| Date.month  | 14              | 15, 18, 22              |
| Date.year   | 13              | 14, 18                  |
| Day.day     | 31              | 35, 38, 39              |
| Day.month   | 32              | 38, 39, 41, 44          |
| Month.month | 63              | 67, 71, 89, 90          |
| Month.year  | 64              | 79, 92, 95              |
| Year.year   | 101             | 104, 108, 110, 112, 117 |

See Table 9.2 for a detailed view of the order in which lines in Listing 9.5 are executed.



# Define/Use Paths in Listing 9.5

- Date.day: 15, 30, 31, 32, 4, 17, 18
- Date.month: 14, 62, 63, 64, 15
- Date.year: 13, 100, 101, 14
- Day.day: 31, 32, 4, 17, 18, 66, 67, 18, 34
- Day.month: 32, 4, 17, 18, 66, 67, 18, 34, 35
- Month.month: 63, 64, 15, 30, 31, 32, 4, 17, 18, 66, 67
- Month.year: No uses in this integration. However, there would be if the month February was used.
- Year.year: 101, 14, 62, 63, 64, 15, 30, 31, 32, 4, 17, 18, 66, 67, 18, 34, 35, 18, 103, 104



# Slice-based Testing

- The concept of program slices was first proposed in 1979.
- Program slices mimic the way programmers, and especially maintenance programmers, think.
- Slices focus on portions of code that are of interest to some question.
- We continue with the same context we had for Define/Use Testing:
- A set  $V$  of variables in a program  $P$  with program graph  $G(P)$ . Usually the set  $V$  contains a single variable,  $v$ )
- There are four distinctions among slices..
  - Forward slices
  - Backward slices
  - Static slices
  - Dynamic slices



# Slice Definitions

- Given a program  $P$  and a set  $V$  of variables in  $P$ , a ***backward slice on the variable set  $V$  at statement  $n$*** , written  $S(V, n)$ , is the set of all statement fragments in  $P$  that contribute to the values of variables in  $V$  at node  $n$ .
- Given a program  $P$  and a set  $V$  of variables in  $P$ , a ***forward slice on the variable set  $V$  at statement  $n$*** , written  $S(V, n)$ , is the set of all statement fragments in  $P$  that are affected by the values of variables in  $V$  at node  $n$ .



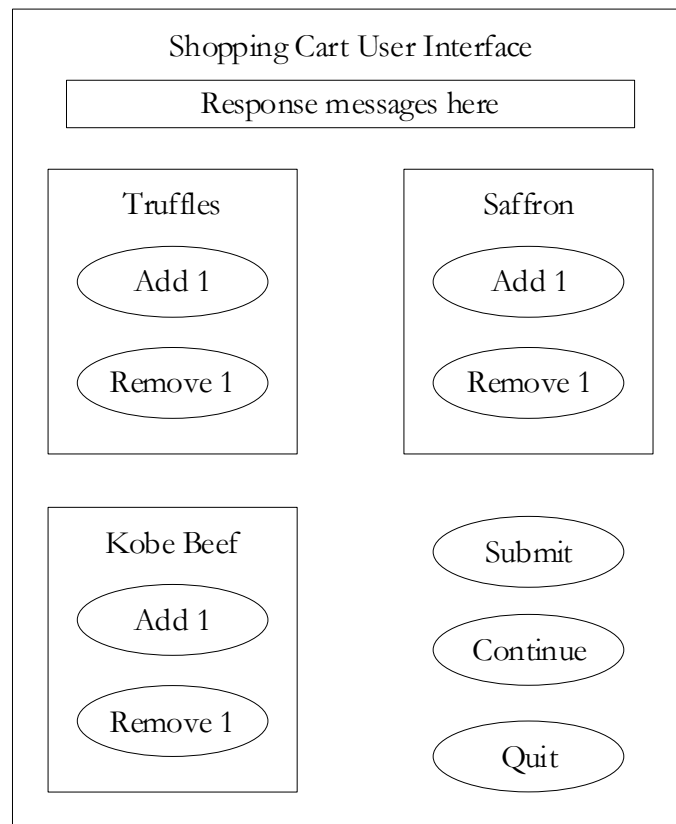
# Static and Dynamic Slices

- The static/dynamic distinction is very analogous to...
  - The difference between the intension and extensions of a database.
  - Variables at compile time versus variable values at execution time.
- These distinctions are improved if we require that a program slice can be compiled and executed.
- In a true tongue twister definition, we can speak about combining separately developed and tested slices: Slice Splicing.



# Slice Example

- The example in section 9.6.1 uses a reduced Foodieltem database and analyzes slices in the updateShoppingCart() method.



# The updateShoppingCart() method.

- Source code and program graph are in Figure 9.7 (too large to include here)
- The text in Chapter 9 shows the following slices...
  - S(totalTruffles, 36)
  - S(totalTruffles, 27)
  - S(totalTruffles, 18)
- Figure 9.8 shows the program graph of the slice on truffleSales at line 39. (next slide)
- We will list slices by sequences of statement fragment numbers.



```

1  private static double updateShoppingCart() {
2      double trufflePrice = 12.50;
5      double truffleSales = 0.0;
8      double totalSales = 0.0;
9      int totalTruffles = 0;
12     String message = "";

13     String event;

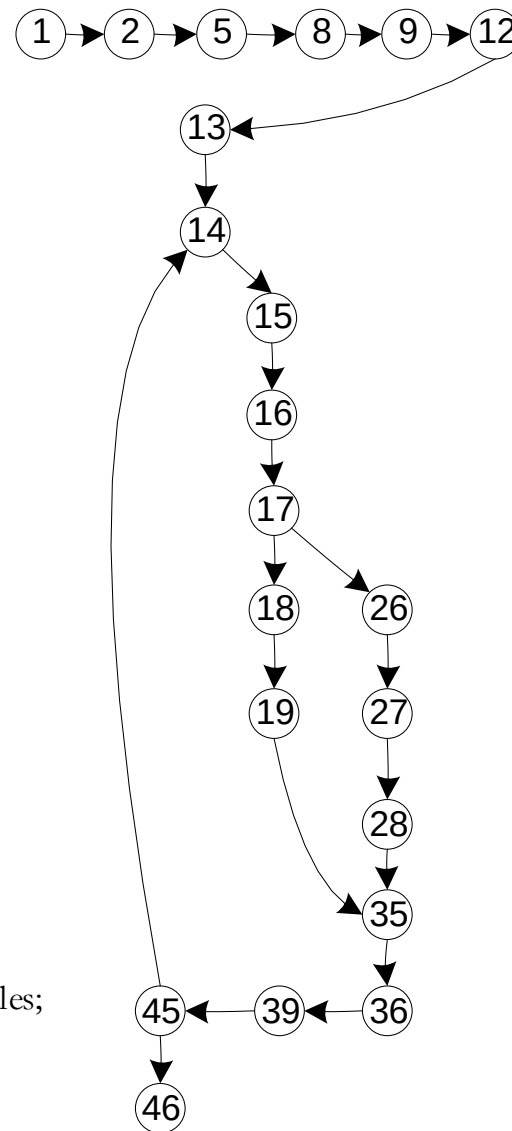
14     do {
15         while(queueOfEvents.isEmpty());
16         event = queueOfEvents.remove();
17         if(event.equalsIgnoreCase("Remove 1 Truffles")
18             && totalTruffles > 0) {
18             totalTruffles--;
19             message = "Removed 1 Truffle";

26         } else if(event.equalsIgnoreCase("Add 1 Truffles")) {
27             totalTruffles++;
28             message = "Added 1 Truffle";
35         }

36         truffleSales = trufflePrice * totalTruffles;
39         totalSales = truffleSales + kobeBeefSales + saffronSales;

45     }
46 }

```





# Interesting updateShoppingCart() Slices.

- $S(\text{totalSales}, 40) =$   
 $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25$   
 $26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40\}$
- $S(\text{truffleSales}, 39) =$   
 $\{1, 2, 5, 8, 9, 12, 13, 14, 15, 16, 17, 18, 19, 26, 27, 28, 35, 36, 39\}$
- $S(\text{truffleSales}, 36) =$   
 $\{1, 2, 5, 8, 9, 12, 13, 14, 15, 16, 17, 18, 19, 26, 27, 28, 35, 36\}$
- $S(\text{totalTruffles}, 27) =$   
 $\{1, 2, 5, 8, 9, 12, 13, 14, 15, 16, 17, 18, 19, 26, 27, 35, 36\}$
- $S(\text{totalTruffles}, 18) = \{1, 2, 5, 8, 9, 12, 13, 14, 15, 16, 17, 18, 35, 36\}$



# Set Relationships Among Slices.

- If we look closely at the slices, we see....
  - $S(\text{totalTruffles}, 18) \dot{\subset} S(\text{totalTruffles}, 27)$
  - $S(\text{totalTruffles}, 27) \dot{\subset} S(\text{totalTruffles}, 36)$
  - $S(\text{totalTruffles}, 36) \dot{\subset} S(\text{truffleSales}, 39)$
  - $S(\text{truffleSales}, 39) \dot{\subset} S(\text{totalSales}, 40)$
- We can find similar chains of slice subsets for
  - totalKobeBeefs
  - totalSaffrons
- Taken together, these form a lattice of slices (Figure 9.9 on next slide)



# Lattice of Slices



# Testing with a Lattice of Slices.

- Recall we suggested that a slice can be compiled and executed.
- If testing reveals a problem with the totalSales variable, we can start with the slices at the leaves of the lattice, test each slice, until we reach the root of the lattice.
- This roadmap can also serve as the order in which agile components can be created and spliced together.
- (Slice splicing strikes again!)



# Program Slicing Tools

| Tool/Product | Language | Static/ dynamic? |
|--------------|----------|------------------|
| Kamkar       | Pascal   | dynamic          |
| Spyder       | ANSI C   | dynamic          |
| Unravel      | ANSI C   | static           |
| CodeSonar®   | C, C++   | static           |
| Indus/Kaveri | java     | static           |
| JSlice       | java     | dynamic          |
| SeeSlice     | C        | dynamic          |



